Nonlinear models for regression

Alberto Suárez
Machine Learning Group
Grupo de aprendizaje automático (GAA)
Computer Science Department
Escuela Politécnica Superior
Universidad Autónoma de Madrid

alberto.suarez@uam.es

Generalized linear models

Typically,
$\phi_0(\mathbf{x}) = 1$ (bias term)

$$h(\mathbf{x};\, \boldsymbol{w}) = \boldsymbol{w}^T \boldsymbol{\phi}(\mathbf{x}) = w_0\phi_0(\mathbf{x}) + w_1\phi_1(\mathbf{x}) + \cdots + w_M\phi_M(\mathbf{x})$$

$$w_0\phi_0(\mathbf{x}_1) + w_2\phi_1(\mathbf{x}_1) + \cdots + w_M\phi_M(\mathbf{x}_1) = y_1$$

$$w_0\phi_0(\mathbf{x}_2) + w_2\phi_1(\mathbf{x}_2) + \cdots + w_M\phi_M(\mathbf{x}_2) = y_2$$

$$\cdots$$

$$w_0\phi_0(\mathbf{x}_N) + w_2\phi_1(\mathbf{x}_N) + \cdots + w_M\phi_M(\mathbf{x}_N) = y_N$$

Design matrix
$\Phi_{nm} = \phi_m(\mathbf{x}_n)$
$n = 1, \ldots, N$
$m = 0,1, \ldots, M$

$$\Rightarrow \begin{pmatrix} \phi_0(\mathbf{x}_1) & \phi_1(\mathbf{x}_1) & \cdots & \phi_M(\mathbf{x}_1) \\ \phi_0(\mathbf{x}_2) & \phi_1(\mathbf{x}_2) & \cdots & \phi_M(\mathbf{x}_2) \\ \vdots & & \ddots & \vdots \\ \phi_0(\mathbf{x}_N) & \phi_1(\mathbf{x}_N) & \cdots & \phi_M(\mathbf{x}_N) \end{pmatrix} \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_M \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

$$\boldsymbol{\Phi}\, \boldsymbol{w} = \mathbf{y} \quad \Rightarrow \quad \boldsymbol{\Phi}^T \boldsymbol{\Phi}\, \boldsymbol{w}^* = \boldsymbol{\Phi}^T \mathbf{y} \Rightarrow \qquad \boldsymbol{w}^* = (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{y}$$

Polynomial regression in 1 D

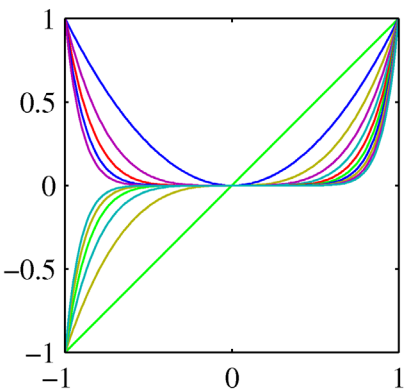
$$h(x;\, \boldsymbol{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M$$

$$w_0\, 1 + w_2 x_1 \;\; + w_2 x_1^2 + \cdots + w_M x_1^M = y_1$$

$$w_0\, 1 + w_2 x_2 \;\; + w_2 x_2^2 + \cdots + w_M x_2^M = y_2$$

$$\cdots$$

$$w_0\, 1 + w_2 x_N \;\; + w_2 x_N^2 + \cdots + w_M x_N^M = y_N$$



Van der Monde matrix
$\mathrm{V}_{nm} = x_n^m$;
$n = 1, \ldots, N$
$m = 0, 1, \ldots, M$

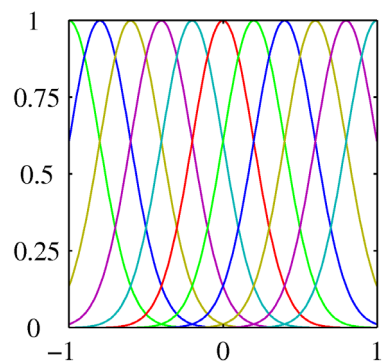
$$\Rightarrow \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^M \\ 1 & x_2 & x_2^2 & \cdots & x_2^M \\ & \vdots & & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^M \end{pmatrix} \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_M \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

$$\mathbf{V}\,\boldsymbol{w} = \mathbf{Y} \;\; \Rightarrow \;\; \mathbf{V}^T\,\mathbf{V}\,\boldsymbol{w}^* = \mathbf{V}^T\,\mathbf{Y} \Rightarrow \qquad \boldsymbol{w}^* = (\mathbf{V}^T\,\mathbf{V})^{-1}\mathbf{V}^T\,\mathbf{Y}$$

Sets of basis functions

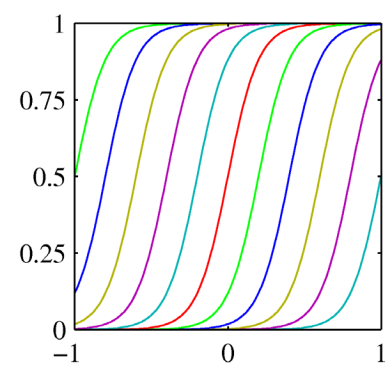
- Fourier basis (sinusoidal functions)
- Wavelets
- Spline basis (piecewise polynomials; usually of degree 3)
- Gaussian basis:

$$\phi_j(\mathbf{x}) = exp\left(-\frac{\|\mathbf{x}-\boldsymbol{\mu}_j\|^2}{s^2}\right)$$



- Sigmoidal basis:

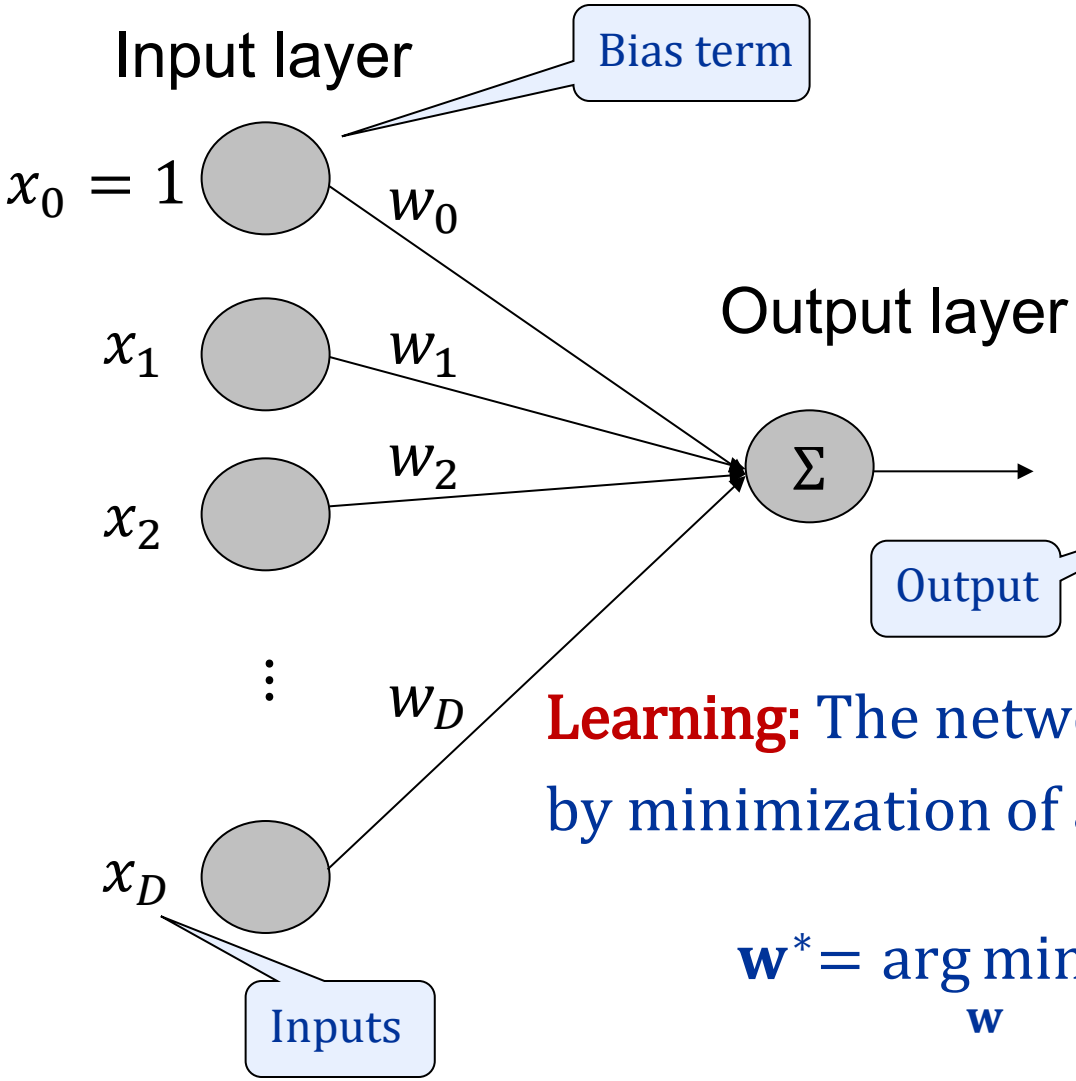
$$\phi_j(\mathbf{x}) = \sigma\left(\frac{\|\mathbf{x}-\boldsymbol{\mu}_j\|}{s}\right) = \frac{1}{1+exp\left(-\frac{\|\mathbf{x}-\boldsymbol{\mu}_j\|}{s}\right)}$$



Fixed vs adaptive sets of basis functions

- Using a fixed set of basis is problematic: Due to the curse of dimensionality, the number of basis functions grows rapidly (often exponentially) with the dimensionality of the input space (D) and the number of inputs (N)
- In many learning problems:
  - As a result of strong correlations between the inputs, $\{\mathbf{x}_n\}_{n=1}^{N}$ can lie close to a nonlinear manifold whose intrinsic dimensionality is smaller than the input space
  - Targets may have significant dependence on only a small number of possible directions within the data manifold.
- Possible solutions:
  - Feature construction: Carefully crafted features by experts.
  - Directly learn the basis functions and dependencies from the data themselves (e.g. Gaussian processes, neural networks).

Single-layer (linear) perceptron for regression



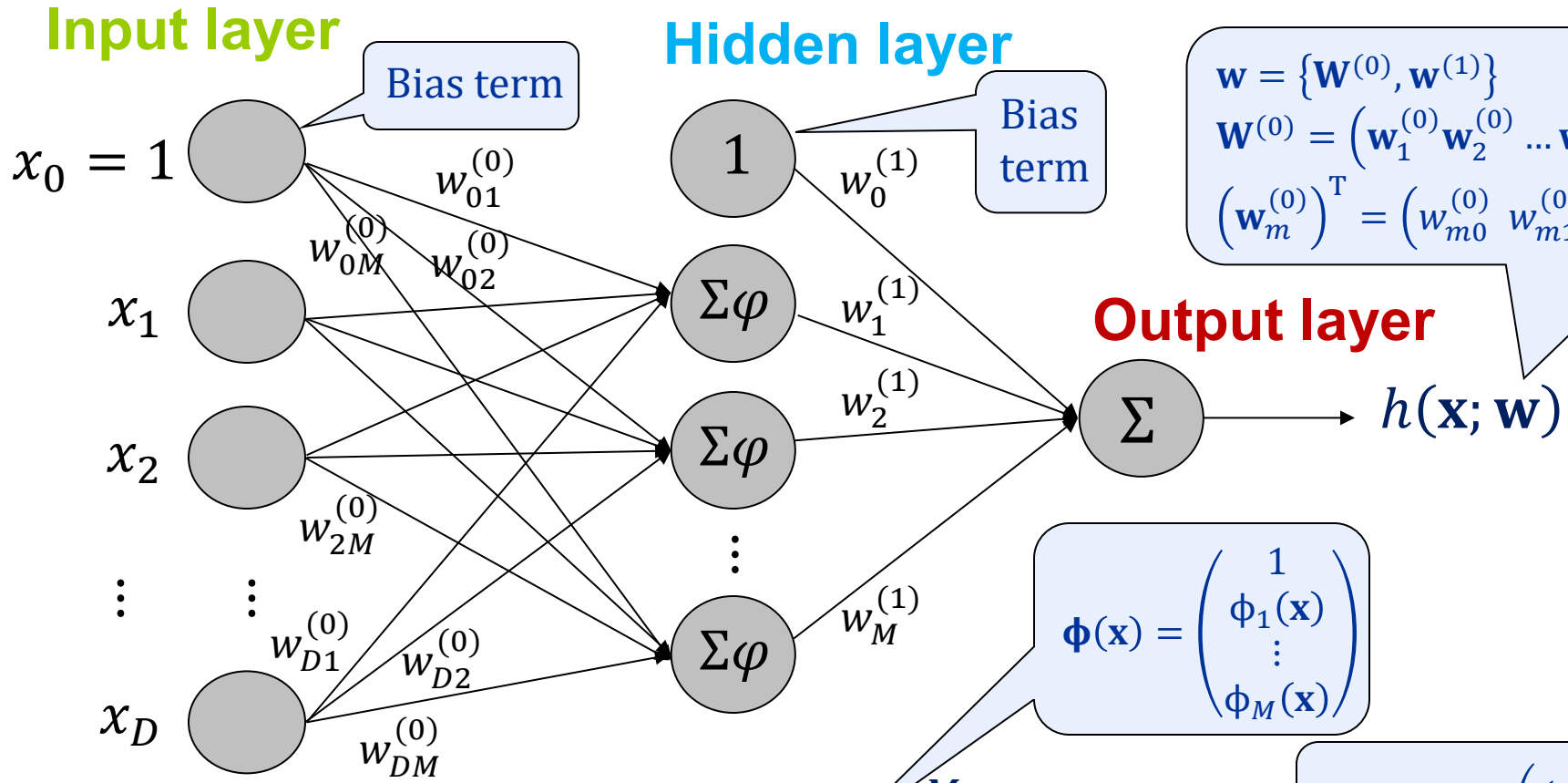
$$\mathbf{w}^{\mathrm{T}} = (w_0, w_1, \dots, w_D)$$
$$\mathbf{x}^{\mathrm{T}} = (x_0\ x_1 \dots x_D)$$

$$h(\mathbf{x}; \mathbf{w}) = \sum_{d=0}^{D} w_d x_d = \mathbf{w}^{\mathrm{T}}\mathbf{x}$$

Learning: The network weights are determined by minimization of a cost function. For instance,

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \sum_{n=1}^{N} (y_n - \mathbf{w}^T \mathbf{x}_n)^2$$

Multi-layer perceptron for regression



$$\mathbf{w} = \{\mathbf{W}^{(0)}, \mathbf{w}^{(1)}\}$$

$$\mathbf{W}^{(0)} = \left(\mathbf{w}_1^{(0)}\mathbf{w}_2^{(0)} \dots \mathbf{w}_M^{(0)}\right)$$

$$\left(\mathbf{w}_m^{(0)}\right)^{\mathrm{T}} = \left(w_{m0}^{(0)}\ w_{m1}^{(0)} \dots w_{mD}^{(0)}\right)$$

$$\boldsymbol{\phi}(\mathbf{x}) = \begin{pmatrix} 1 \\ \phi_1(\mathbf{x}) \\ \vdots \\ \phi_M(\mathbf{x}) \end{pmatrix}$$

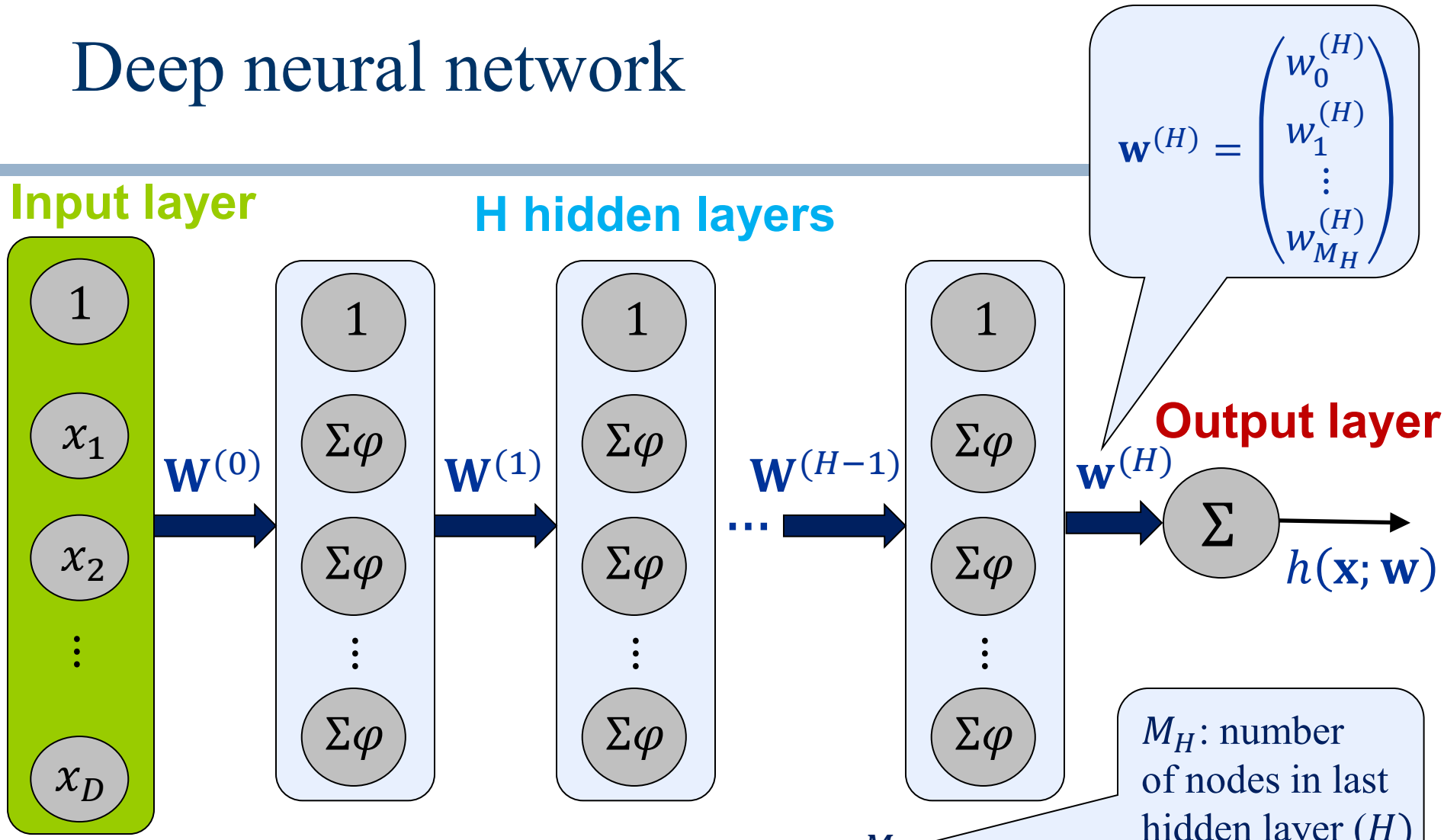
$$h(\mathbf{x};\mathbf{w}) = \left(\mathbf{w}^{(1)}\right)^T \boldsymbol{\phi}(\mathbf{x}) = \sum_{m=0}^{M} w_m^{(1)} \phi_m(\mathbf{x})$$

$$\phi_m(\mathbf{x}) = \varphi\left(\left(\mathbf{w}_m^{(0)}\right)^{\mathrm{T}} \mathbf{x}\right)$$

$$\left(\mathbf{w}^{(1)}\right)^{\mathrm{T}} = \left(w_1^{(1)}\ w_2^{(1)} \dots w_M^{(1)}\right)$$

$$\phi_0(\mathbf{x}) = 1$$

Deep neural network



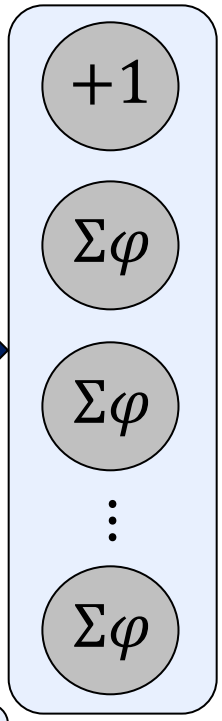
$$h(\mathbf{x};\mathbf{w}) = \left(\mathbf{w}^{(H)}\right)^T \boldsymbol{\phi}^{(H)}(\mathbf{x}) = \sum_{m=0}^{M_H} w_m^{(H)} \phi_m^{(H)}(\mathbf{x})$$

$\mathbf{w} = \{\mathbf{W}^{(0)}, \mathbf{W}^{(1)}, \dots, \mathbf{W}^{(H-1)}, \mathbf{w}^{(H)}\}$

Output of hidden layer h

$$\mathbf{W}^{(h-1)} = \left(\mathbf{w}_1^{(h-1)} \mathbf{w}_2^{(h-1)} \dots \mathbf{w}_{M_h}^{(h-1)}\right)$$

$$\boldsymbol{\phi}^{(h-1)}(\mathbf{x}) \xrightarrow{\mathbf{W}^{(h-1)}}$$



$$\rightarrow \boldsymbol{\phi}^{(h)}(\mathbf{x}) = \begin{pmatrix} 1 \\ \phi_1^{(h)}(\mathbf{x}) \\ \vdots \\ \phi_{M_h}^{(h)}(\mathbf{x}) \end{pmatrix}$$

$$\phi_0^{(h)} = 1$$

$$\boldsymbol{\phi}^{(0)}(\mathbf{x}) = \mathbf{x} = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_D \end{pmatrix}$$

$$M_0 = D$$

$$\boldsymbol{\phi}^{(h-1)}(\mathbf{x}) = \begin{pmatrix} 1 \\ \phi_1^{(h-1)}(\mathbf{x}) \\ \vdots \\ \phi_{M_{h-1}}^{(h-1)}(\mathbf{x}) \end{pmatrix}$$

$$\left(\mathbf{w}_m^{(h-1)}\right)^{\mathrm{T}} = \left(w_{0m}^{(h-1)} \; w_{1m}^{(h-1)} \dots w_{M_{h-1}m}^{(h-1)}\right)$$

$$\phi_m^{(h)}(\mathbf{x}) = \varphi^{(h)}\left(\left(\mathbf{w}_m^{(h-1)}\right)^{\mathrm{T}} \boldsymbol{\phi}^{(h-1)}(\mathbf{x})\right); \qquad m = 1, \dots, M_h$$

$$M_{H+1} = 1$$

In the last layer: $\varphi^{(H+1)}(z) = z$ (linear activation)

$$h = 1, \dots, H, H+1$$

Sigmoid activation function

$$\sigma(z) = \frac{1}{1+e^{-z}}; \qquad logit(p) = \sigma^{-1}(p) = \log\frac{p}{1-p}$$

Also: transfer function

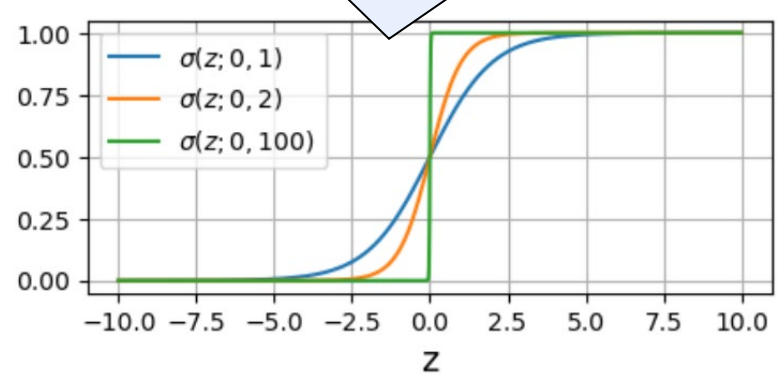
- $\sigma(-\infty) = 0; \quad \sigma(0) = \frac{1}{2}; \sigma(+\infty) = 1;$
- Monotonically increasing: $z_2 > z_1 \Rightarrow \sigma(z_2) > \sigma(z_1)$
- Symmetry: $\sigma(-z) = 1 - \sigma(z)$
- Derivative: $\sigma'(z) = \frac{d\sigma(z)}{dz} = \sigma(z)\big(1-\sigma(z)\big)$

Also in output layer for classification (logistic regression)

$$\sigma(z; z_0, b) = \frac{1}{1+e^{-b(z-z_0)}}$$

$\frac{1}{b}$: scale

z_0: center



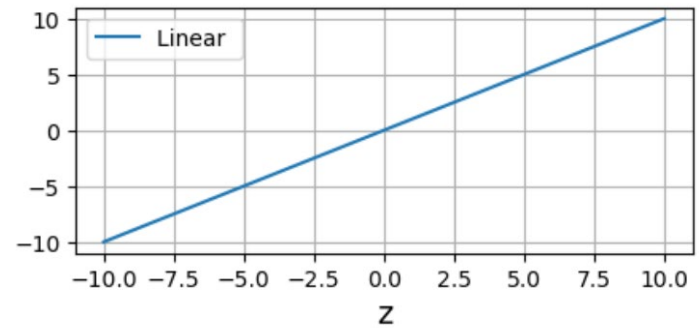
$$\lim_{b\to\infty} \sigma(z; z_0, b) = \begin{cases} 0 & \text{if } z < z_0 \\ \frac{1}{2} & \text{if } z = z_0 \\ 1 & \text{if } z > z_0 \end{cases}$$

Heavyside step function

Other activation functions

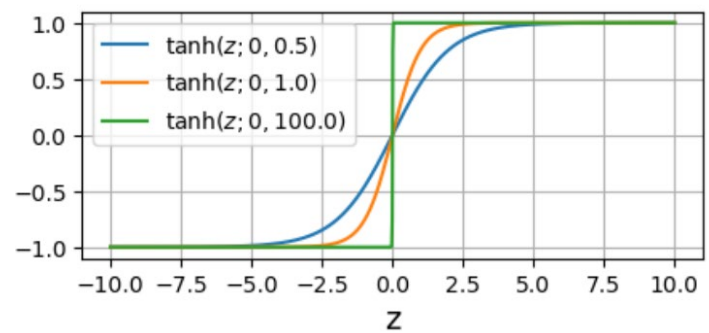
- **Linear**: $\varphi(z) = z$

Also in output layer for regression



- **Hyperbolic tangent**:

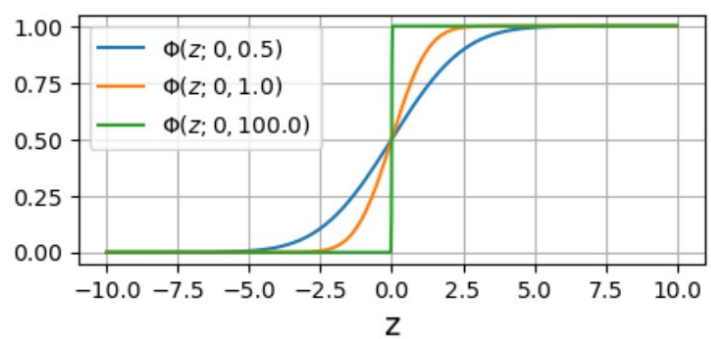
$$\tanh(z; z_0, b) = \frac{e^{b(z-z_0)} - e^{-b(z-z_0)}}{e^{b(z-z_0)} + e^{-b(z-z_0)}}$$



normcdf

- **Probit**: $\Phi(z; z_0, b) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{b(z-z_0)} e^{-\frac{y^2}{2}} dy$

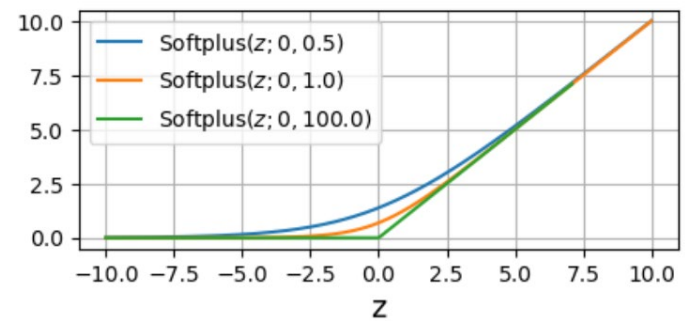
Also in output layer for classification (probit regression)



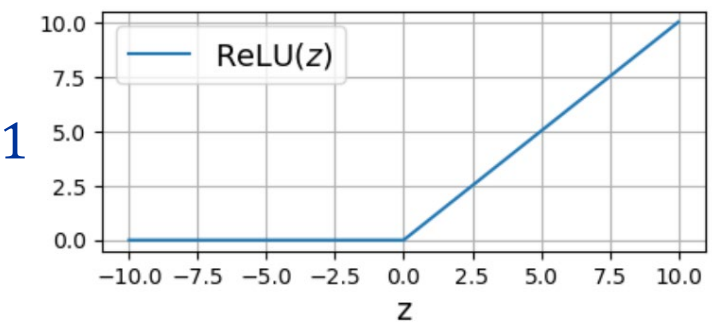
Other activation functions

- **softplus**$(z; z_0, b) = \frac{log(1+e^{b(z-z_0)})}{b}$

$$\lim_{b\to\infty} \text{softplus}(z; 0, b) = \text{ReLU}(z)$$



- **Rectified linear unit**: $\text{ReLU}(z) = \max(0, z)$

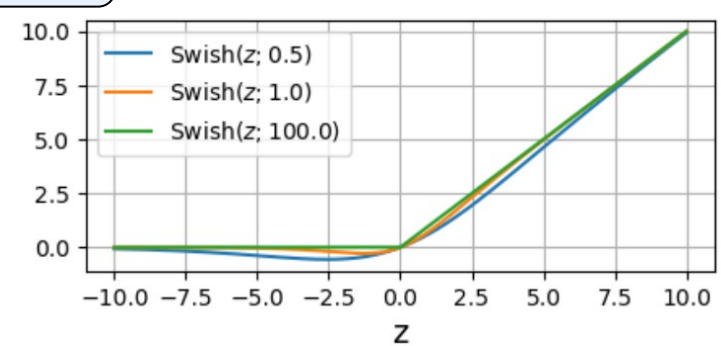


- **Leaky ReLU**$(z) = \begin{cases} az & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$; $0 < a \ll 1$

Avoids zero gradient if $z < 0$

E.g. $a = 0.01$

- **Swish**$(z;\ b) = z\,\sigma(z; 0, b) = z\frac{1}{1+e^{-bz}}$



Learning the parameters of a predictive model

$\boldsymbol{w}$: Model parameters

Consider the predictive model:

$$y = h(\mathbf{x};\ \boldsymbol{w}) + \epsilon$$

$h(\mathbf{x};\ \boldsymbol{w})$: Deterministic, real-valued function of the attributes [H1]

ϵ : Additive iid noise, independent of the attributes [H2]

- $\mathrm{E}[\epsilon] = 0$
- $\epsilon \perp \mathbf{x}$
- $\epsilon \sim f_\epsilon(\epsilon; \boldsymbol{\theta})$

$\boldsymbol{\theta}$: Parameters of the noise distribution

- We have a set of labeled data $\{(\mathbf{x}_n, y_n)\}_{n=1}^N$ to calibrate the model.

  These data are iid samples from the predictive model [H3]

$$y_n = h(\mathbf{x}_n;\ \boldsymbol{w}) + \epsilon_n; \qquad n = 1,2,\ldots,N$$

Least squares

- Predictive model

$$y_n = h(\mathbf{x}_n;\ \boldsymbol{w}) + \epsilon_n; \qquad n = 1,2,\dots,N$$

- Mean squared error

$$MSE(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N}\left(y_n - h(\mathbf{x}_n;\boldsymbol{w})\right)^2$$

- Least squares: Find the parameters that minimize MSE

$$\boldsymbol{w}_{LS} = \arg\min_{\boldsymbol{w}} \sum_{n=1}^{N}\left(y_n - h(\mathbf{x}_n;\boldsymbol{w})\right)^2$$

Minimize MAE

- Predictive model

$$y_n = h(\mathbf{x}_n;\ \boldsymbol{w}) + \epsilon_n; \qquad n = 1,2,\dots,N$$

- Mean absolute error

$$MAE(\boldsymbol{w}) = \frac{1}{N}\sum_{n=1}^{N} |y_n - h(\mathbf{x}_n;\ \boldsymbol{w})|$$

- Find the parameters that minimize MAE

$$\boldsymbol{w}_{MAE} = \arg\min_{\boldsymbol{w}} \sum_{n=1}^{N} |y_n - h(\mathbf{x}_n;\ \boldsymbol{w})|$$

Maximum likelihood

- The likelihood of the model, given the data $\{(\mathbf{x}_n, y_n)\}_{n=1}^N$ is

$$\mathcal{L}(\boldsymbol{w}, \boldsymbol{\theta}) = P(\{y_n\}_{n=1}^N \big| \{\mathbf{x}_n\}_{n=1}^N; \boldsymbol{w}, \boldsymbol{\theta})$$

Data are iid samples

$$= \prod_{n=1}^N P(y_n | \mathbf{x}_n; \boldsymbol{w}, \boldsymbol{\theta})$$

$\epsilon_n = y_n - h(\mathbf{x}_n;\ \boldsymbol{w})$

$$= \prod_{n=1}^N f_\epsilon(y_n - h(\mathbf{x}_n;\ \boldsymbol{w}); \boldsymbol{\theta})$$

- Numerically, it is preferable to maximize the log-likelihood

$$\log \mathcal{L}(\boldsymbol{w}, \boldsymbol{\theta}) = \sum_{n=1}^N log f_\epsilon(y_n - h(\mathbf{x}_n;\ \boldsymbol{w}); \boldsymbol{\theta})$$

$$\boldsymbol{w}_{ML}, \boldsymbol{\theta}_{ML} = \arg\max_{\boldsymbol{w},\boldsymbol{\theta}}(\log \mathcal{L}(\boldsymbol{w}, \boldsymbol{\theta})) = \arg\min_{\boldsymbol{w},\boldsymbol{\theta}}(-\log \mathcal{L}(\boldsymbol{w}, \boldsymbol{\theta}))$$

Maximum likelihood: Gaussian noise

- $\epsilon \sim N(0, \sigma)$ $\qquad f_\epsilon(\epsilon; \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{\epsilon^2}{2\sigma^2}}$

$$-\log \mathcal{L}(\boldsymbol{w}, \sigma^2) = -\sum_{n=1}^{N} log\left(\frac{1}{\sqrt{2\pi\sigma^2}} exp\left(-\frac{1}{2\sigma^2}\left(y_n - h(\mathbf{x}_n;\ \boldsymbol{w})\right)^2\right)\right)$$

$$= \frac{N}{2} log(2\pi) + \frac{N}{2} log\sigma^2 + \frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(y_n - h(\mathbf{x}_n;\ \boldsymbol{w})\right)^2$$

$$\frac{\partial}{\partial\sigma^2}\left(-\log \mathcal{L}(\boldsymbol{w}, \sigma^2)\right)\Big|_{\boldsymbol{w}_{ML}, \sigma^2_{ML}} = \frac{N}{2\sigma^2} - \frac{1}{2\sigma^4}\sum_{n=1}^{N}\left(y_n - h(\mathbf{x}_n;\ \boldsymbol{w})\right)^2\Bigg|_{\boldsymbol{w}_{ML}, \sigma^2_{ML}} = 0$$

$$\sigma^2_{ML} = \frac{1}{N}\sum_{n=1}^{N}\left(y_n - h(\mathbf{x}_n;\ \boldsymbol{w}_{ML})\right)^2;$$

Mean squared error

$$\boldsymbol{w}_{ML} = \arg\min_{\boldsymbol{w}} \sum_{n=1}^{N}\left(y_n - h(\mathbf{x}_n; \boldsymbol{w})\right)^2$$

Least squares!

Maximum likelihood: Laplace noise

- $\epsilon \sim f_\epsilon(\epsilon; b) = \frac{1}{2b} e^{-\frac{|\epsilon|}{b}}$

$$-\log \mathcal{L}(\boldsymbol{w}, b) = -\sum_{n=1}^{N} log\left(\frac{1}{2b} e^{-\frac{|\epsilon|}{b}}\right)$$

$$= N\, log(2) + N\, logb + \frac{1}{b}\sum_{n=1}^{N} |y_n - h(\mathbf{x}_n;\ \boldsymbol{w})|$$

$$\frac{\partial}{\partial \sigma^2}(-\log \mathcal{L}(\boldsymbol{w}, b))\Big|_{\boldsymbol{w}_{ML}, \sigma^2_{ML}} = \frac{N}{b} - \frac{1}{b^2}\sum_{n=1}^{N} |y_n - h(\mathbf{x}_n;\ \boldsymbol{w})|\Big|_{\boldsymbol{w}_{ML}, b_{ML}} = 0$$

$$b_{ML} = \frac{1}{N}\sum_{n=1}^{N} |y_n - h(\mathbf{x}_n; \boldsymbol{w}_{ML})|\,;$$

Mean absolute error

$$\boldsymbol{w}_{ML} = \arg\min_{\boldsymbol{w}} \sum_{n=1}^{N} |y_n - h(\mathbf{x}_n;\ \boldsymbol{w})|$$

Minimize MAE

Maximum posterior

- Prior: $\boldsymbol{w} \sim f_{\boldsymbol{w}}(\boldsymbol{w}; \boldsymbol{\gamma})$
- The likelihood of the model, given the data $\{(\mathbf{x}_n, y_n)\}_{n=1}^N$ is

$$P\left(\boldsymbol{w} \middle| \{(\mathbf{x}_n, y_n)\}_{n=1}^N; \boldsymbol{\theta}, \boldsymbol{\gamma}\right) = \frac{P\left(\{y_n\}_{n=1}^N \middle| \{\mathbf{x}_n\}_{n=1}^N; \boldsymbol{w}, \boldsymbol{\theta}\right) P(\boldsymbol{w}|\boldsymbol{\gamma})}{P\left(\{y_n\}_{n=1}^N \middle| \{\mathbf{x}_n\}_{n=1}^N; \boldsymbol{\theta}, \boldsymbol{\gamma}\right)}$$

- Maximization of the posterior

$$\boldsymbol{w}_{MAP} = \arg\max_{\boldsymbol{w}} \left(P\left(\boldsymbol{w} \middle| \{(\mathbf{x}_n, y_n)\}_{n=1}^N; \boldsymbol{\theta}, \boldsymbol{\gamma}\right)\right)$$

$$= \arg\max_{\boldsymbol{w}} \left(P\left(\{y_n\}_{n=1}^N \middle| \{\mathbf{x}_n\}_{n=1}^N; \boldsymbol{w}, \boldsymbol{\theta}\right) P(\boldsymbol{w}|\boldsymbol{\gamma})\right)$$

$$= \arg\min_{\boldsymbol{w}} \left(-\log P\left(\{y_n\}_{n=1}^N \middle| \{\mathbf{x}_n\}_{n=1}^N; \boldsymbol{w}, \boldsymbol{\theta}\right) - \log P(\boldsymbol{w}|\boldsymbol{\gamma})\right)$$

MAP: Gaussian noise + Gaussian prior

- Prior: $\boldsymbol{w} \sim N(0, \sigma_{\boldsymbol{w}}^2)$; $\quad f_{\boldsymbol{w}}(\boldsymbol{w}; \sigma_{\boldsymbol{w}}^2) = \frac{1}{\left((2\pi)^D \sigma_{\boldsymbol{w}}^2\right)^{1/2}} e^{-\frac{\boldsymbol{w}^{\mathrm{T}}\boldsymbol{w}}{2\sigma_{\boldsymbol{w}}^2}}$

$$-\log f_{\boldsymbol{w}}(\boldsymbol{w}; \sigma_{\boldsymbol{w}}^2) = \frac{D}{2} log(2\pi) + \frac{1}{2} log\sigma_{\boldsymbol{w}}^2 + \frac{1}{2\sigma_{\boldsymbol{w}}^2} \boldsymbol{w}^{\mathrm{T}}\boldsymbol{w}$$

$$\boldsymbol{w}_{MAP} = \arg\min_{\boldsymbol{w}} \left[\sum_{n=1}^{N} \left(y_n - h(\mathbf{x}_n; \boldsymbol{w})\right)^2 + \frac{\sigma^2}{\sigma_{\boldsymbol{w}}^2} \boldsymbol{w}^{\mathrm{T}}\boldsymbol{w} \right]$$

L_2 penalty

MAP: Gaussian noise + Laplace prior

- Prior: $\boldsymbol{w} \sim f_{\boldsymbol{w}}(\boldsymbol{w}; b_{\boldsymbol{w}})$; $\quad f_{\boldsymbol{w}}(\boldsymbol{w}; b_{\boldsymbol{w}}) \propto e^{-\frac{\|\boldsymbol{w}\|_1}{b_{\boldsymbol{w}}}}$

$$-\log f_{\boldsymbol{w}}(\boldsymbol{w}; b_{\boldsymbol{w}}) = constant + \frac{1}{b_{\boldsymbol{w}}} \|\boldsymbol{w}\|_1$$

$$\boldsymbol{w}_{ML} = \arg\min_{\boldsymbol{w}} \left[\sum_{n=1}^{N} \left(y_n - h(\mathbf{x}_n; \boldsymbol{w}) \right)^2 + \frac{2\sigma^2}{b_{\boldsymbol{w}}} \|\boldsymbol{w}\|_1 \right]$$

L_1 penalty

Learning the network parameters

The network weights are determine by minimization of a cost function.

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} C(\mathbf{w})$$

- Assuming $C(\mathbf{w})$ is differentiable twice, the conditions for minimization:
  - **Zero gradient**: $\left.\frac{\partial C(\mathbf{w})}{\partial \mathbf{w}}\right|_{\mathbf{w}=\mathbf{w}^*} = \mathbf{0}$;
  - **Positive definite Hessian**: Hessian matrix evaluated at minimum

$$\left\{ H_{ij}^* = \left.\frac{\partial C(\mathbf{w})}{\partial w_i \partial w_j}\right|_{\mathbf{w}=\mathbf{w}^*} ; i,j = 1, \ldots dim(\mathbf{w}) \right\}$$ has positive eigenvalues

Gradient descent for least squares

The network weights are determine by minimization of a cost function.

- Cost function: $C(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}(h(\mathbf{x}_n; \boldsymbol{w}) - y_n)^2$
- Gradient: $\frac{\partial C(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{2}\sum_{n=1}^{N}\frac{\partial}{\partial \mathbf{w}}(h(\mathbf{x}_n; \boldsymbol{w}) - y_n)^2 = \sum_{n=1}^{N}\delta_n \frac{\partial h(\mathbf{x}_n;\mathbf{w})}{\partial \mathbf{w}}$
- Gradient descent:

Learning rate: $\eta > 0$

Error: $\delta_n = h(\mathbf{x}_n; \mathbf{w}) - y_n$
backpropagation

$$\mathbf{w}^{[0]}$$

$$\mathbf{w}^{[k+1]} = \mathbf{w}^{[k]} - \eta \left.\frac{\partial C(\mathbf{w})}{\partial \mathbf{w}}\right|_{\mathbf{w}=\mathbf{w}^{[k]}}; \quad k = 0,1,2,\ldots$$

Epoch loop

Convergence: $\mathbf{w}^* = \lim_{k\longrightarrow\infty} \mathbf{w}^{[k]}$

$\mathbf{w}^{[0]}$: seed
$\mathbf{w}^{[k]}$: estimation at epoch k

Error Backpropagation [Wabos 1974, 1994] [Rumelhart, Hinton, & Williams,1986]

Batch learning

- Minimize cost function:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} C(\mathbf{w})$$

- Gradient of the cost function:

$$\frac{\partial C(\mathbf{w})}{\partial \mathbf{w}}$$

seed

$\mathbf{w}^{[0]}$

k $= 0$

Repeat

$$\mathbf{w}^{[k+1]} = \mathbf{w}^{[k]} - \eta \left.\frac{\partial C(\mathbf{w})}{\partial \mathbf{w}}\right|_{\mathbf{w}=\mathbf{w}^{[k]}}$$

k = k + 1 % epoch counter

Until stopping criterion is met

Learning rate: $\eta > 0$

Stopping criteria

- Use error on validation set for **early stopping**,
- **Small change** of $C(\mathbf{w})$.
- Maximum number of epochs reached
- …

Learning with momentum

$\mathbf{w}^{[0]}$

$k = 0$

$\delta\mathbf{w}^{[0]} = 0$

Repeat

$$\delta\mathbf{w}^{[k+1]} = -\eta \left.\frac{\partial C(\mathbf{w})}{\partial \mathbf{w}}\right|_{\mathbf{w}=\mathbf{w}^{[k]}} + \mu\delta\mathbf{w}^{[k]}$$

$$\mathbf{w}^{[k+1]} = \mathbf{w}^{[k]} + \delta\mathbf{w}^{[k+1]}$$

$k = k + 1$ % epoch counter

Until stopping criterion is met

Learning rate: $\eta > 0$

Strength of momentum term: $\mu > 0$

Momentum term: Smoother evolution (not necessarily beneficial)

Online learning

- Cost function: $C(\mathbf{w}) = \sum_{n=1}^{N} C(\mathbf{w}; \mathbf{x}_n, y_n)$
- Gradient of the cost function: $\frac{\partial C(\mathbf{w})}{\partial \mathbf{w}} = \sum_{n=1}^{N} \frac{\partial C(\mathbf{w}; \mathbf{x}_n, y_n)}{\partial \mathbf{w}}$

$\mathbf{w}^{[0]}$
k $= 0$
i $= 0$
Repeat
    for n = 1:N
        $\mathbf{w}^{[i+1]} = \mathbf{w}^{[i]} - \eta \frac{\partial C(\mathbf{w}; \mathbf{x}_n, y_n)}{\partial \mathbf{w}}$
        i = i +1 % iteration counter
    k = k + 1 % epoch counter
Until stopping criterion is met

Stochastic gradient descent

- Cost function: $\mathbf{w}^* = \arg\min_{\mathbf{w}} C(\mathbf{w})$
- Gradient of the cost function: $\frac{\partial C(\mathbf{w})}{\partial \mathbf{w}} = \sum_{n=1}^{N} \frac{\partial C(\mathbf{w};\mathbf{x}_n,y_n)}{\partial \mathbf{w}}$

Hyperparameters:
- $\eta > 0$
- Batch size (N/B)
- Number of iterations

$\mathbf{w}^{[0]}$
k $= 0$
Repeat
  Partition data into B mini-batches: $\{B_1, \dots, B_B\}$
  for b = 1:B
    $\delta C_b\left(\mathbf{w}^{[i]}\right) = \sum_{n \in B_b} \frac{\partial C(\mathbf{w};\mathbf{x}_n,y_n)}{\partial \mathbf{w}}$
    $\mathbf{w}^{[i+1]} = \mathbf{w}^{[i]} - \eta\, \delta C_b\left(\mathbf{w}^{[i]}; \mathbf{x}_n, y_n\right)$
  k = k + 1 % epoch counter
Until stopping criterion is met

Other optimization methods

The network weights are determined by minimization of a cost function.

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} C(\mathbf{w})$$

- Conjugate gradients
- Newton (requires Hessian)
- Quasi-Newton (uses approximations to the Hessian):
  - BFGS: Broyden–Fletcher–Goldfarb–Shanno
  - DFP: Davidon–Fletcher–Powell
- Subgradient methods ($C(\mathbf{w})$ not differentiable)
- …

Approximation to Hessian is positive definite, which guarantees reduction of cost at each iteration.

References

- https://www.3blue1brown.com/topics/neural-networks
- C. M. Bishop "Pattern Recognition and Machine Learning", Springer (2006) [https://www.microsoft.com/en-us/research/people/cmbishop/#!prml-book]
- Carl Edward Rasmussen and Christopher K. I. Williams, "Gaussian Processes for Machine Learning" The MIT Press, 2006. ISBN 0-262-18253-X.[http://www.gaussianprocess.org/gpml/]
- "Deep learning" Ian Goodfellow and Yoshua Bengio and Aaron Courville
  http://www.deeplearningbook.org/
- Scikit learn documentation
  - Generalized linear models
    http://scikit-learn.org/stable/modules/linear_model.html
  - Kernel ridge regression
    http://scikit-learn.org/stable/modules/kernel_ridge.html#m2012
  - Gaussian processes for regression
    http://scikit-learn.org/stable/modules/gaussian_process.html
    http://scikit-learn.org/stable/auto_examples/gaussian_process/plot_gpr_noisy_targets.html
  - Multilayer perceptron
    http://scikit-learn.org/stable/modules/neural_networks_supervised.html
    http://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html